Pattern-Based Dependencies and Agentic Discovery for Data Quality

Khalid Belhajjame

PSL, Université Paris Dauphine, LAMSADE, Paris, France

Data Quality: Beyond Classical Dependencies

Data quality rules are often expressed as dependencies:

- Functional Dependencies (FDs): $X \to Y$
- Conditional FDs (CFDs): value-dependent constraints

However, real-world data exhibits richer regularities:

- Dependencies often hold on *parts of values*
- Patterns such as prefixes, tokens, or substrings are informative

Examples:

- First name $\to$ gender (e.g., "John*" $\to$ M)
- ZIP prefix $\to$ city
- Email domain $\to$ organization

Challenge:

- These dependencies are *not expressible* using standard FDs/CFDs
- They are often *approximate* and hold with noise

Limitations of Functional Dependencies

Functional Dependencies (FDs):

- Express relationships between *entire attribute values*
- Example: `name` $\rightarrow$ `gender`

Example dataset:

name	gender
John Smith	M
John Brown	M
John Taylor	M
Susan Miller	F
Susan Clark	M

Observation:

- FD `name` $\rightarrow$ `gender` is **not violated**
- No two identical names with different genders

However:

- The value "Susan Clark" is likely incorrect
- The dependency holds at the level of **patterns** ("Susan*" $\rightarrow$ F)

Pattern Functional Dependencies (PFDs)

Key idea:

- Dependencies may hold on *parts of attribute values*
- Patterns (prefixes, tokens, substrings) can determine other attributes

Examples:

- `first_name(name)` $\rightarrow$ `gender`
- `prefix(zip, 3)` $\rightarrow$ `city`
- `suffix(email)` $\rightarrow$ `organization`

Comparison:

- FDs: compare **entire values**
- PFDs: compare **patterns extracted from values**

Implication:

- PFDs can detect errors that FDs cannot
- Particularly useful for *string and semi-structured data*

Formal Definition of Pattern Functional Dependencies

Definition:

- A Pattern Functional Dependency (PFD) is defined as:

$$R(X \rightarrow Y, T_p)$$

- R is a relation schema
- $X \rightarrow Y$ is a functional dependency
- T_p is a **pattern tableau**

Pattern tableau T_p:

- Contains **patterns** instead of exact values
- Patterns can be:
  - prefixes (e.g., "John*")
  - substrings or tokens
  - regular expressions

Semantics:

- If two tuples match the same pattern in X,
- then they must match the same pattern in Y

Example of a Pattern Functional Dependency

Relation:

name	gender
John Smith	M
John Brown	M
Susan Miller	F
Susan Clark	F

Pattern Functional Dependency:

$$R(\texttt{name} \rightarrow \texttt{gender}, T_p)$$

Pattern tableau T_p:

name pattern	gender
John*	M
Susan*	F

Interpretation:

- Tuples matching "John*" have gender M
- Tuples matching "Susan*" have gender F

Approximate Pattern Functional Dependencies

Motivation:

- Real-world data is often **noisy**
- Pattern-based dependencies rarely hold perfectly

Example (with noise):

name	gender
John Smith	M
John Brown	M
Susan Miller	F
Susan Clark	M

Observation:

- Pattern "Susan*" $\rightarrow$ F is **mostly true**
- One tuple violates the pattern

Approximate PFD:

- A PFD that holds **with some tolerated violations**
- Evaluated using:
  - **support**
  - **confidence**
  - **noise threshold**

Metrics for Approximate PFDs

Given a candidate rule: $X \to Y$

Support:

- Number of tuples satisfying the pattern on X

$$\text{support}(X) = |\{t \in R \mid t \models X\}|$$

Confidence:

- Fraction of tuples where Y agrees within each group defined by X

$$\text{conf}(X \to Y) = \frac{|\{t \in R \mid t \models X \wedge t \models Y\}|}{|\{t \in R \mid t \models X\}|}$$

Noise / Violation rate:

- Proportion of tuples that violate the dependency

$$\text{noise} = 1 - \text{conf}(X \to Y)$$

Typical thresholds:

- support $\geq K$
- conf $\geq \theta$ (e.g., 0.9)

Overview of PFD Discovery

Goal:

- Discover approximate Pattern Functional Dependencies from a relation R

Main idea:

- Transform attribute values into **patterns**
- Search for dependencies over these patterns

Pipeline:

1. **Pattern extraction**
   - Tokenization, substrings, prefixes
2. **Pattern grouping**
   - Group tuples that match the same pattern
3. **Candidate generation**
   - Generate $X \rightarrow Y$ candidates
4. **Validation**
   - Compute support and confidence
   - Apply thresholds
5. **Generalization**
   - Merge patterns into more general rules

1. Pattern Extraction

Objective:

- Extract candidate **patterns** from attribute values

Given a value: `"John Smith"`

Possible extracted patterns:

- Tokens: `"John"`, `"Smith"`
- Prefixes: `"J"`, `"Jo"`, `"Joh"`, `"John"`
- Substrings / n-grams:
  - `"ohn"`, `"Smi"`, `"mit"`

Key idea:

- Each value generates **many candidate patterns**

Challenge:

- Very large number of patterns
- Many are **irrelevant or spurious**

Solution (in practice):

- Use frequency thresholds
- Index patterns efficiently (e.g., inverted lists)

2. Pattern Grouping

Objective:

- Group tuples that match the same pattern

Example: pattern = "John*"

name	gender
John Smith	M
John Brown	M
John Taylor	M

Result:

- All tuples matching "John*" form a **group**

Key idea:

- Patterns induce a **partition** of the data
- Each group corresponds to a value of X

Use:

- Check whether Y is consistent within each group
- Basis for computing support and confidence

Patterns on Both Sides of a PFD

So far:

- We considered rules of the form:

$$\text{pattern}(X) \rightarrow Y$$

- Example: "John*" $\rightarrow$ M

General case:

- Patterns can be applied to **both** sides:

$$\text{pattern}(X) \rightarrow \text{pattern}(Y)$$

Examples:

- `prefix(zip,3)` $\rightarrow$ `city`
- `domain(email)` $\rightarrow$ `organization`

In practice:

- Most useful rules are:

$$\text{pattern}(X) \rightarrow Y$$

- Easier to interpret and use for data cleaning

3. Candidate Generation

Objective:

- Generate candidate dependencies of the form:

$$X \rightarrow Y$$

From patterns:

- Each attribute may have multiple pattern representations:
  - `name`, `first_token(name)`, `prefix(name,2)`, ...

Example candidates:

- `first_token(name)` $\rightarrow$ `gender`
- `prefix(zip,3)` $\rightarrow$ `city`
- `domain(email)` $\rightarrow$ `organization`

Search space:

- Large number of possible $X \rightarrow Y$
- Can include combinations of attributes

Challenge:

- Avoid exploring all candidates exhaustively

4. Validation of Candidate Dependencies

Objective:

- Determine whether a candidate $X \to Y$ is a valid (approximate) PFD

Step 1: Group tuples by X

- Tuples with the same pattern on X form a group

Step 2: Check consistency of Y

- For each group, check whether Y takes (mostly) the same value

Step 3: Compute metrics

- Support: number of tuples satisfying X
- Confidence: proportion of tuples consistent on Y

Step 4: Apply thresholds

- Keep rule if:
  - support $\geq K$
  - conf $\geq \theta$

Result:

- Valid approximate PFDs

5. Generalization of Patterns

Objective:

- Merge specific patterns into more general and meaningful rules

Example (specific rules):

- "John*" $\rightarrow$ M
- "James*" $\rightarrow$ M

Generalized rule:

- `first_token(name)` $\rightarrow$ `gender`

Why generalize?

- Reduce redundancy
- Improve interpretability
- Capture broader regularities

Trade-off:

- More general rules may have:
  - lower confidence
  - higher coverage

Running Example

Relation R:

name	zip	city
John Smith	90012	Los Angeles
John Brown	90013	Los Angeles
Susan Miller	10001	New York
Susan Clark	10002	New York
Alice Green	60601	Chicago

Goal:

- Discover approximate Pattern Functional Dependencies

Expected patterns:

- `prefix(zip,3)` $\rightarrow$ `city`
- `first_token(name)` $\rightarrow$ `city` (unlikely)

Step 1: Pattern Extraction

Extract patterns from attribute values

name = "John Smith"

- `first_token(name)` = John

zip = 90012

- `prefix(zip,3)` = 900
- `prefix(zip,2)` = 90

Result: enriched table

name	zip	prefix(zip,3)	city
John Smith	90012	900	Los Angeles
John Brown	90013	900	Los Angeles
Susan Miller	10001	100	New York
Susan Clark	10002	100	New York
Alice Green	60601	606	Chicago

Step 2: Pattern Grouping

Group tuples by pattern: `prefix(zip,3)`

Groups:

- 900 $\rightarrow$ {Los Angeles, Los Angeles}
- 100 $\rightarrow$ {New York, New York}
- 606 $\rightarrow$ {Chicago}

Observation:

- Each group has consistent city values

Candidate rule:

$$\texttt{prefix(zip,3)} \rightarrow \texttt{city}$$

Step 3: Validation

Evaluate: `prefix(zip,3)` $\rightarrow$ `city`

Support:

- 5 tuples

Confidence:

- All groups are consistent
- $\Rightarrow$ confidence $= 1.0$

Conclusion:

- Valid PFD

Counter-example candidate:

`first_token(name)` $\rightarrow$ `city`

- "John" $\rightarrow$ Los Angeles (OK)
- "Susan" $\rightarrow$ New York (OK)
- "Alice" $\rightarrow$ Chicago (OK)

Limitations of Classical PFD Discovery

1. **Large search space of patterns**
   - Many possible transformations:
     - prefixes, suffixes, substrings, tokens, n-grams
   - Example: `prefix(zip,1)`, `prefix(zip,2)`, `prefix(zip,3)`, ...
2. **No semantic understanding**
   - Algorithm explores patterns syntactically
   - Does not know which patterns are meaningful
3. **Many spurious dependencies**
   - Some rules hold by coincidence in the data
   - Example: `first_token(name)` $\rightarrow$ `city`
4. **Parameter sensitivity**
   - Results depend on:
     - pattern types considered
     - thresholds (support, confidence)

Key limitation:

- Search is driven by **syntax**, not **semantics**

Why Agentic AI for PFD Discovery?

Key idea:

- Complement algorithmic search with **semantic reasoning**

What classical algorithms lack:

- Understanding of data semantics
- Ability to prioritize meaningful patterns
- Ability to interpret and validate rules

What agentic AI provides:

- **Semantic profiling**
  - Identify attribute types (name, zip, email, etc.)
- **Guided pattern selection**
  - Suggest relevant transformations (prefix, domain, token)
- **Search prioritization**
  - Focus on promising candidates
- **Rule interpretation**
  - Explain and filter discovered dependencies

Goal:

- Discover **fewer but more meaningful** dependencies

Agentic Workflows for PFD Discovery

Goal:

- Combine algorithms and agentic AI to improve PFD discovery

Three main workflows:

1. **Feature-Enriched Discovery**
   - Agent suggests meaningful transformations
   - Algorithm performs full discovery
2. **Guided Search**
   - Agent prioritizes promising candidates
   - Algorithm validates selected rules
3. **Agent-in-the-Loop Discovery**
   - Agent proposes, evaluates, and refines rules
   - Iterative interaction with the algorithm

Key difference:

- Increasing role of the agent in the discovery process

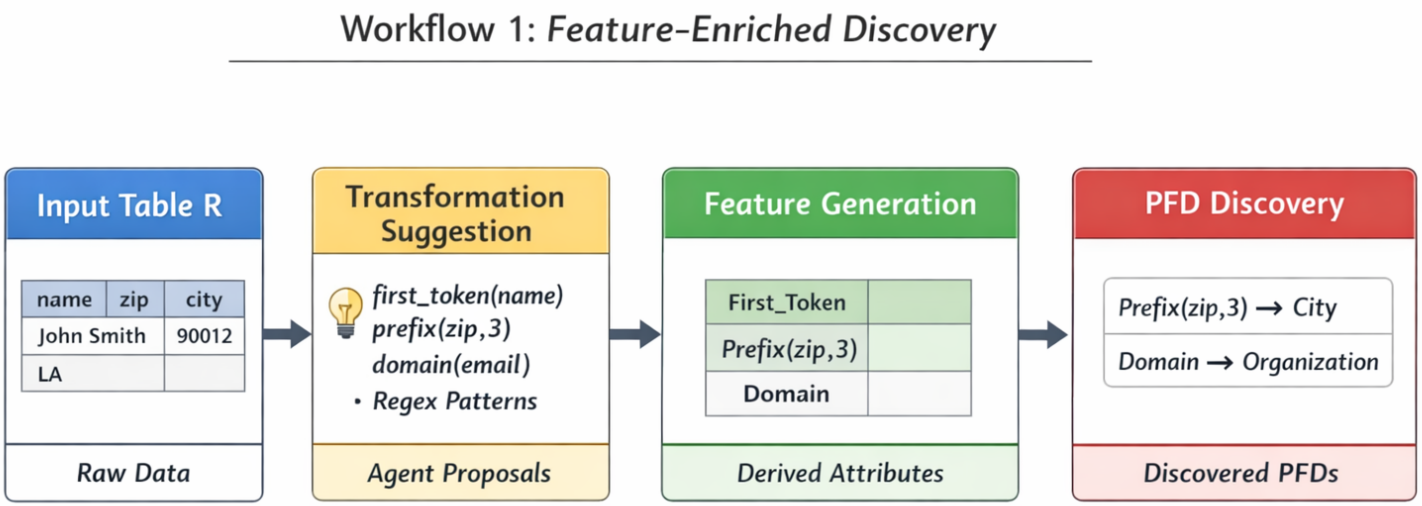
Workflow 1: Feature-Enriched Discovery

Key idea:

- Use agentic AI to suggest meaningful transformations
- Then apply the classical PFD discovery algorithm

Separation of roles:

- Agent: **feature engineering**
- Algorithm: **dependency discovery**



Workflow 1: Step 1 — Transformation Suggestion

Input:

- Relation schema
- Sample values

Agent task:

- Identify attribute semantics:
  - name, zip, email, phone, code, etc.
- Suggest relevant transformations:
  - `first_token(name)`
  - `prefix(zip,3)`
  - `domain(email)`

Output:

- Set of transformation functions

Workflow 1: Step 2 — Feature Generation

Objective:

- Apply transformations to create new attributes

Example:

name	zip	city
John Smith	90012	LA

After feature generation:

name	prefix(zip,3)	first_token(name)	city
John Smith	900	John	LA

Workflow 1: Step 3 — PFD Discovery

Input:

- Enriched table with derived attributes

Process:

- Apply standard PFD discovery:
  - candidate generation
  - grouping
  - validation (support, confidence)

Output:

- Discovered PFDs
  - `prefix(zip,3)` $\rightarrow$ `city`
  - `domain(email)` $\rightarrow$ `organization`

Workflow 2: Guided Search

Key idea:

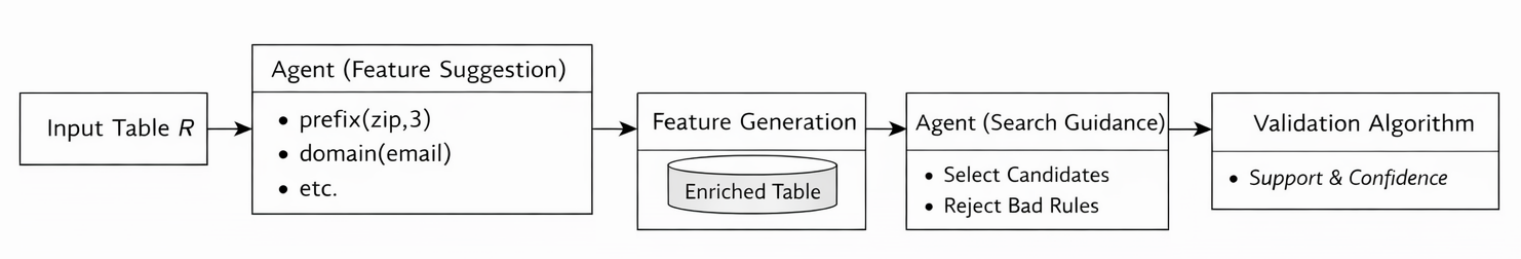
- Use agentic AI to **guide the search for dependencies**
- Reduce the number of candidates explored

Difference with Workflow 1:

- Agent does not only suggest features
- Agent also **prioritizes candidate dependencies**

Separation of roles:

- Agent: **search guidance**
- Algorithm: **validation (support, confidence)**



Workflow 2: Step 1 — Feature Space

Same as Workflow 1:

- Apply transformations to generate derived attributes

Example:

name	prefix(zip,3)	first_token(name)	city
John Smith	900	John	LA
Susan Miller	100	Susan	NY

Result:

- A set of candidate features (patterns)

Key difference starts next:

- Instead of exploring all dependencies,
- we **guide the search**

Workflow 2: Step 2 — Candidate Selection

Objective:

- Select promising candidate dependencies $X \to Y$

Agent task:

- Analyze attribute semantics
- Suggest meaningful dependencies

Examples:

- `prefix(zip,3)` $\to$ `city`
- `domain(email)` $\to$ `organization`

Avoid:

- Exploring unlikely candidates
- Example: `first_token(name)` $\to$ `city`

Result:

- Reduced set of candidate dependencies

Workflow 2: Step 3 — Validation

Input:

- Candidate dependencies proposed by the agent

Algorithm task:

- Validate each candidate:
  - grouping
  - support
  - confidence

Decision:

- Keep rules that satisfy thresholds

Output:

- Discovered approximate PFDs

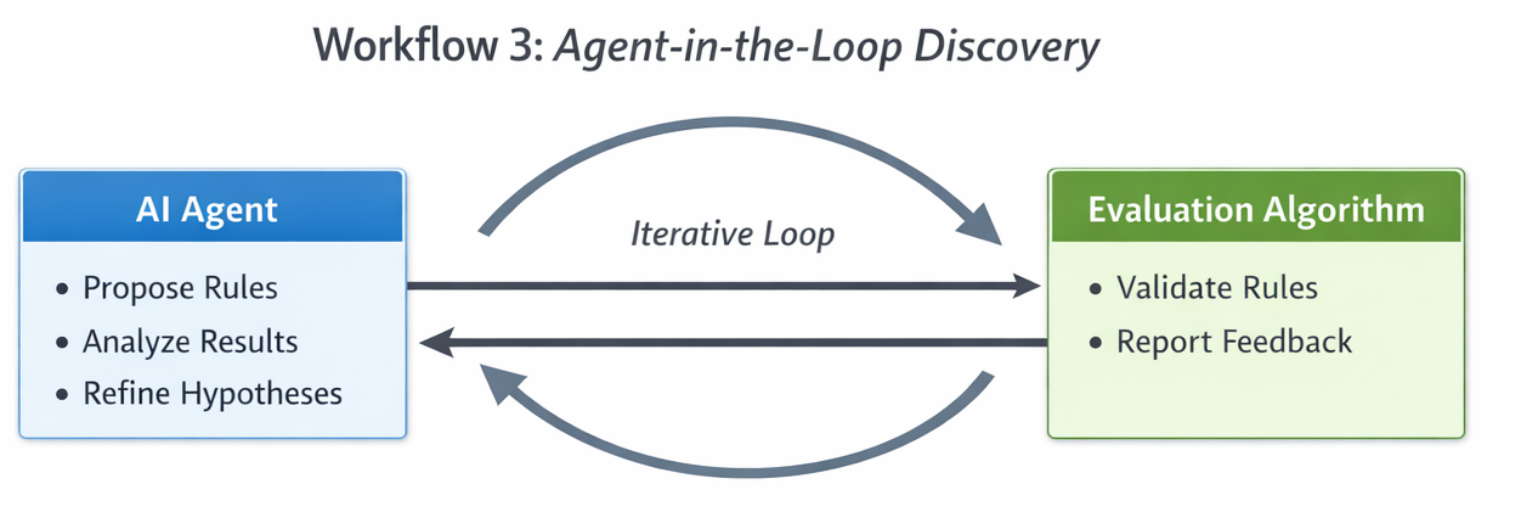
Workflow 3: Agent-in-the-Loop Discovery

Key idea:

- Integrate the agent in an **iterative discovery loop**

Separation of roles:

- Agent:
  - propose rules
  - analyze results
  - refine hypotheses
- Algorithm:
  - validate rules (support, confidence)



Workflow 3: Step 1 — Hypothesis Generation

Agent task:

- Propose initial candidate dependencies

Based on:

- attribute names
- sample values
- domain knowledge

Examples:

- `prefix(zip,3)` $\rightarrow$ `city`
- `domain(email)` $\rightarrow$ `organization`

Workflow 3: Step 2 — Validation and Feedback

Algorithm:

- Validate candidate rules:
  - support
  - confidence

Output:

- Valid rules
- Violations

Feedback to agent:

- Which rules are strong
- Which rules fail

Workflow 3: Step 3 — Refinement

Agent task:

- Refine rules based on feedback

Examples:

- Try `prefix(zip,2)` instead of `prefix(zip,3)`
- Replace pattern by another transformation

Goal:

- Improve:
  - confidence
  - coverage

Process:

- Iterate until convergence

Student Project: Objectives

Goal of the project:

- Study the discovery of approximate Pattern Functional Dependencies (PFDs)
- Compare a **classical fully algorithmic approach** with an **agentic AI-enhanced approach**

Students will work in groups of 4–5 and must:

- Implement the classical PFD discovery algorithm
- Implement at least **one** of the proposed agentic workflows
- Run experiments on provided datasets
- Compare the approaches quantitatively and qualitatively

Expected outcome:

- Understand the strengths and limitations of classical and agentic discovery
- Assess whether agentic AI improves the quality or efficiency of discovery

Student Project: Required Tasks

Task 1: Classical PFD Discovery

- Implement the classical algorithm:
  - pattern extraction
  - candidate generation
  - validation (support, confidence)

Task 2: Agentic Workflow

- Implement at least **one** workflow:
  - Feature-Enriched Discovery, or
  - Guided Search, or
  - Agent-in-the-Loop Discovery

Task 3: Experiments

- Run both approaches on the provided datasets
- Compare results and performance

Datasets

Datasets will be provided

Characteristics:

- Real-world datasets with string attributes
- Contain meaningful patterns (names, zip codes, emails, etc.)
- May include noise and inconsistencies

Examples:

- ZIP code – city datasets
- Email – organization datasets
- Name – gender datasets

Important:

- Use the same datasets for all methods
- Ensure fair comparison between approaches

Experimental Evaluation

You must evaluate:

1. **Discovered dependencies**
   - Number of PFDs discovered
   - Quality and interpretability of rules
2. **Metrics**
   - Support
   - Confidence
3. **Performance**
   - Execution time
   - Number of candidates explored (if possible)

Goal:

- Assess trade-offs between:
  - quality of rules
  - computational cost

Use of Multiple LLMs

Agentic components use LLMs

Requirement (when possible):

- Compare at least two LLMs

Examples:

- Mistral
- Claude
- (others if available)

Objective:

- Evaluate differences in:
  - suggested transformations
  - candidate selection
  - final discovered rules

Deliverables

1. **Implementation**
   - Code for classical PFD discovery
   - Code for at least one agentic workflow
2. **Report**
   - Description of your approach
   - Experimental results
   - Comparison between methods
   - Discussion and insights
3. **Demonstration (15mn)**
   - Presentation of your system
   - Explanation of key design choices
   - Discussion of results